

Energy Efficiency

Prakrit Parajuli, Alea Schröder, Dirk Wübben, Armin Dekorsy

Dept. of Communications Engineering, University of Bremen, Bremen, Germany

Email: {parajuli, schroeder, wuebben, dekorsy}@ant.uni-bremen.de

Abstract—Low Earth Orbit (LEO) satellite-to-handheld connections herald a new era in satellite communications. Space-Division Multiple Access (SDMA) precoding is a method that mitigates interference among satellite beams, boosting spectral efficiency. While optimal SDMA precoding solutions have been proposed for ideal channel knowledge in various scenarios, addressing robust precoding with imperfect channel information has primarily been limited to simplified models. However, these models might not capture the complexity of LEO satellite applications. We use the Soft Actor-Critic (SAC) deep Reinforcement Learning (RL) method to learn robust precoding strategies without the need for explicit insights into the system conditions and imperfections. Our results show flexibility to adapt to arbitrary system configurations while performing strongly in terms of achievable rate and robustness to disruptive influences compared to analytical benchmark precoders.

Index Terms—6G, Multi-user beamforming, 3D networks, Low Earth Orbit (LEO), Satellite Communications, Machine Learning (ML), deep Reinforcement Learning (RL)

I. INTRODUCTION

The upcoming sixth generation standard of mobile communications will seek to extend our classic terrestrial networks to so-called 3D networks by integrating communication satellites and Unmanned Aerial Vehicles (UAVs) [1]. This additional degree of freedom is expected to boost continuous global coverage, improve balancing traffic demand surges, and increase outage protection [2], [3]. In contrast to higher satellite orbits, the Low Earth Orbit (LEO) offers relatively low latency and path loss, and reduced deployment cost. Therefore, LEO satellites are targeted as a key component of future Non Terrestrial Networks (NTN) [1]. The satellites' downlink transmit power can be steered by Space-Division Multiple Access (SDMA) precoding. The precoder optimizes each users' signal power while mitigating inter-user interference, achieving better spectral efficiency [4]. Conventional SDMA precoding techniques, however, are challenged by errors in position estimation caused by outdated Channel State Information at Transmitter (CSIT), which can degrade the performance quickly [5]. Under real circumstances, a variety of perturbing effects may further influence the transmission quality. They are induced by, for example, the high relative velocity of LEO satellites and atmospheric influences.

In order to design a robust precoder, the authors in [5] maximize the achievable rate, taking imperfect position knowledge into account, and propose a low complexity algorithm using

supervised learning. In [6], an analytical robust precoder is derived that utilizes the second order statistics of the channel to maximize the mean Signal-to-Leakage-and-Noise Ratio (SLNR). [7] investigate Rate-Splitting Multiple Access (RSMA) to deal with errors in position measurements, by splitting the user messages into individually precoded common and private parts. In our previous paper [8], we take a first look at using deep Reinforcement Learning (RL) for the purpose of beamforming in the presence of imperfect CSIT. Deep Learning (DL), as a sub-set of Machine Learning (ML), infers and iteratively tunes the parameters of a Neural Network (NN) based on a data set such that the NN approximately maximizes an objective. These data-driven DL approaches have achieved noteworthy performance across most domains in the past decade, and their application in communication networks has been under intense study, e.g., [9]. ML is of particular interest for robust algorithms since input data with uncertainty are often seen as desirable and positive during the learning process [10, Chp. 7.5]: to prevent over-fitting on the available data, practitioners will frequently add noise on their data as a regularizing measure. In [8], we assume a multi-satellite downlink scenario and use RL to train a NN that takes as input a channel matrix estimate to output a precoding matrix that approximately maximizes the achievable sum rate. We use the Soft Actor-Critic (SAC) RL algorithm [11] that allows for continuous-valued output and maintains a measure of uncertainty about its decisions. This measure of uncertainty is used to guide the learning process to be sample efficient. We demonstrate the feasibility of this approach and its strong robustness to disruptive influences comparing to the common Minimum Mean Squared Error (MMSE) precoding approach.

In this paper we move on to a single satellite downlink scenario that is more challenging due to covering much larger user distances. We compare our performance to the popular MMSE precoder, as well as the above mentioned analytical robust precoder [6], both of which strongly favor this spatially decoupled scenario. In Section II and Section III, we first discuss the satellite downlink model and the two benchmark precoders. Afterwards, we outline the learning approach in Section IV and discuss the adjustments for this more complex scenario. Finally, we present our findings in Section V and conclude in Section VI.

Notations: Boldface $\mathbf{x}$ and $\mathbf{X}$ denote vectors resp. matrices. $\mathbf{I}_N$ is an $N \times N$ identity matrix. We use the operators Transpose $\{\cdot\}^T$, Hermitian $\{\cdot\}^H$, Expectation $\mathbb{E}\{\cdot\}$, Hadamard product $\circ$, absolute value $|\cdot|$ and Euclidean norm $\|\cdot\|$.

This work was partly funded by the German Ministry of Education and Research (BMBF) under grant 16KISK016 (Open6GHub) and 16KIS1028 (MOMENTUM) and the European Space Agency (ESA) under contract number 4000139559/22/UK/AL (AICoMS).

II. DOWNLINK SATELLITE COMMUNICATION MODEL

We examine a single-satellite multi-user downlink scenario as depicted in Fig. 1. The LEO satellite is equipped with a Uniform Linear Array (ULA) consisting of N antennas with an inter-antenna-distance d_n and transmit gain G_{Sat} . The K users are assumed to be handheld devices with just one receive antenna and low receive gain G_{Utr} . The Line-of-Sight (LoS) channel $\mathbf{h}_k \in \mathbb{C}^{1 \times N}$ between the satellite and user k is modeled by

$$\mathbf{h}_{k,m}(\nu_{k,m}) = \frac{\lambda \sqrt{G_{\text{Utr}} G_{\text{Sat}}}}{4\pi d_{k,m}} e^{-j\varphi_{k,m}} \mathbf{v}_{k,m}(\cos(\nu_{k,m})), \quad (1)$$

$$\mathbf{h}_k(\nu_k) = \frac{1}{\sqrt{\text{PL}_k}} e^{-j\kappa_k} \mathbf{v}_k(\cos(\nu_k)). \quad (2)$$

The path loss PL_k is the linear representation of the free space path loss FSPL_k influenced by large scale fading $\text{LF}_k \sim \mathcal{N}(0, \sigma_{\text{LF}}^2)$ with $\text{PL}_k^{\text{dB}} = \text{FSPL}_k^{\text{dB}} + \text{LF}_k^{\text{dB}}$. The linear free space path loss FSPL_k for a given wavelength λ and a satellite-to-user distance d_k is

$$\text{FSPL}_k = \frac{16\pi^2 d_k^2}{\lambda^2 G_{\text{Utr}} G_{\text{Sat}}}. \quad (3)$$

The overall phase shift from the satellite to user k corresponds to $\kappa_k \in [0, 2\pi]$, while the relative phase shifts from the N satellite antennas to user k are determined by the steering vector $\mathbf{v}_k \in \mathbb{C}^{1 \times N}$. The n -th entry of the steering vector $\mathbf{v}_k(\cos(\nu_k))$ calculates as follows

$$v_k^n(\cos(\nu_k)) = e^{-j\pi \frac{d_n}{\lambda} (N+1-2n) \cos(\nu_k)}, \quad (4)$$

where ν_k is the Angle of Departure (AoD) from the satellite to user k . Under real circumstances, the estimate of the AoDs at the satellite might be flawed. We model this behavior as a uniformly distributed additive error $\varepsilon_k \sim \mathcal{U}(-\Delta\varepsilon, +\Delta\varepsilon)$ on the space angles $\cos(\nu_k)$. In [8], we show that this error can be interpreted as an overall multiplicative error $\mathbf{v}_k(\varepsilon_{k,m}) \in \mathbb{C}^{1 \times N}$ on our channel vector $\mathbf{h}_k(\nu_k)$:

$$\tilde{\mathbf{h}}_k(\nu_k, \varepsilon_k) = \mathbf{h}_k(\nu_k) \circ \mathbf{v}_k(\varepsilon_k). \quad (5)$$

In order to perform SDMA, we calculate a precoding vector $\mathbf{w}_k \in \mathbb{C}^{N \times 1}$ for each user k based on this estimate of the channel vector $\tilde{\mathbf{h}}_k \in \mathbb{C}^{1 \times N}$. Different approaches to determine the precoding vectors are discussed in the subsequent sections. After calculating the precoding vector $\mathbf{w}_k$, the data symbol s_k of each user k is weighted with this precoding vector $\mathbf{w}_k$. Taking complex Additive White Gaussian Noise (AWGN) $n_k \sim \mathcal{CN}(0, \sigma_n^2)$ into account, the received signal y_k for user k results in

$$y_k = \mathbf{h}_k \mathbf{w}_k s_k + \mathbf{h}_k \sum_{l \neq k}^K \mathbf{w}_l s_l + n_k. \quad (6)$$

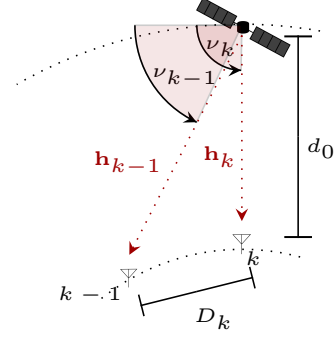


Fig. 1. The single-satellite downlink scenario. The satellite is positioned at an altitude d_0 . Two users $k, k-1$, are positioned at AoDs ν_k, ν_{k-1} . They are characterized by their channel vectors $\mathbf{h}_k, \mathbf{h}_{k-1}$ and their inter-user distance D_{Utr} .

From (6), we get the Signal-to-Interference-plus-Noise Ratio (SINR) Γ_k for user k

$$\Gamma_k = \frac{|\mathbf{h}_k \mathbf{w}_k|^2}{\sigma_n^2 + \sum_{l \neq k}^K |\mathbf{h}_k \mathbf{w}_l|^2}. \quad (7)$$

The achievable rate R , equal to the sum rate, is given by

$$R = \sum_{k=1}^K \log_2(1 + \Gamma_k) \quad (8)$$

and serves as the performance metric for the different precoding approaches in this paper. In summary, our goal is to maximize the expected sum rate in presence of imperfect positional knowledge (5) by learning a robust precoding algorithm using the SAC technique.

III. CONVENTIONAL PRECODERS

This section introduces 1) a conventional non-robust MMSE precoding approach and 2) a robust SLNR precoder based on the second order statistics of the channel considering imperfect position knowledge. These analytical precoders serve as a benchmark for the learned precoders.

A. MMSE

The MMSE precoder is a well established precoder for scenarios with perfect channel state information [12] and will serve as a baseline comparison in this work. For a channel estimation $\tilde{\mathbf{H}} = [\tilde{\mathbf{h}}_1 \dots \tilde{\mathbf{h}}_K]^T$ the corresponding MMSE precoding matrix $\mathbf{W}^{\text{MMSE}} = [\mathbf{w}_1^{\text{MMSE}} \dots \mathbf{w}_K^{\text{MMSE}}]$ is given as

$$\mathbf{W}^{\text{MMSE}} = \sqrt{\frac{P}{\text{tr}\{\mathbf{W}'^H \mathbf{W}'\}}} \cdot \mathbf{W}', \quad (9)$$

$$\mathbf{W}' = [\tilde{\mathbf{H}}^H \tilde{\mathbf{H}} + \sigma_n^2 \frac{K}{P} \mathbf{I}_N]^{-1} \tilde{\mathbf{H}}^H$$

where P denotes the overall transmit power of the satellite. We highlight that the MMSE approach is not always optimal in terms of the sum rate R (8).

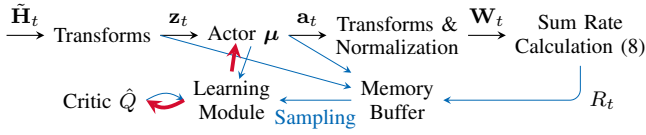


Fig. 2. The learned SAC precoders' process flow. Black arrows form an inference step, blue arrows show the components that are collected for a learning step, and red arrows show NN parameter updates.

IV. REINFORCEMENT LEARNED PRECODER

Our goal in this section will be to find a function that takes as input the estimated channel matrix $\tilde{\mathbf{H}}$ (5) and outputs a precoding matrix $\mathbf{W}$ that maximizes the expectation $\bar{R}$ of the achievable sum rate R (8). We will use a parameterized deep NN μ_{θ_μ} , subsequently called Actor Neural Network (AcNN), as a model function and then use the SAC [11] algorithm to tune this network's parameters θ_μ . SAC assumes the true system dynamics are unknown, e.g., the distribution of errors on the estimated CSIT $\tilde{\mathbf{H}}$. Therefore, it trains a second NN $\hat{Q}_{\theta_{\hat{Q}}}$ that has parameters $\theta_{\hat{Q}}$ in parallel to approximate the mapping of $(\tilde{\mathbf{H}}, \mathbf{W}) \rightarrow \bar{R}$. This known function $\hat{Q}_{\theta_{\hat{Q}}}$, subsequently referred to as the Critic Neural Network (CrNN), is used to as a guide to tune the precoding AcNN μ_{θ_μ} . SAC recommends the use of multiple, independently initialized CrNN for stability [11]. In the following, we will omit the parameter indices θ in AcNN $\mu_{\theta_\mu} \equiv \mu$ and CrNN $\hat{Q}_{\theta_{\hat{Q}}} \equiv \hat{Q}$ for readability.

Learning to optimize the NN parameters via SAC is comprised of two independent components: 1) data generation; 2) parameter updates. Fig. 2 gives an overview of the process flow, we describe it in more detail in the following. First, to generate the data to learn from, we perform an inference step t , starting by obtaining a complex-valued channel state estimate $\tilde{\mathbf{H}}_t \in \mathbb{C}^{K \times N}$. We then flatten it into a vector and transform it to a real vector $\tilde{\mathbf{z}} \in \mathbb{R}^{1 \times 2KN}$ as it is easier for NN to digest. For the real-complex transformation we consider decomposition into a) real and imaginary part, as well as b) decomposition into magnitude and phase, the choice of which we will discuss briefly later. The inputs $\tilde{\mathbf{z}}$ are also standardized to approximately zero mean and unit variance using static scaling factors, which is known to promote convergence speed in NN [10]. We discuss this choice in Section V-A. We forward the standardized vector $\mathbf{z}_t \in \mathbb{R}^{1 \times 2KN}$ through the AcNN, which is a standard feed-forward NN with four times as many outputs as we require entries for a precoding matrix $\mathbf{W}$. The outputs are grouped in pairs of two, where one of each pair represents the mean and the other the scale of a Normal distribution to sample from. This formulation gives us a measure of uncertainty about each output that we will make use of during the training step. After sampling from the distributions given by the output pairs, we transform this output $\mathbf{a} \in \mathbb{R}^{1 \times 2KN}$ into a complex vector of half length and reshape to a precoding matrix. Finally, we rescale the matrix to the available signal power and gain a normalized precoding matrix $\mathbf{W}_t \in \mathbb{C}^{N \times K}$. In testing, we find that magnitude-phase decomposition works best for the network input and real-imaginary composition works best for the output, though

this is still under investigation. To conclude a data generation step, we evaluate the sum rate R_t achieved by this precoding matrix $\mathbf{W}_t$ and store the tuple of $(\mathbf{z}, \mathbf{a}, R)_t$ in a memory buffer for the learning step.

Learning steps are performed using the Stochastic Gradient Descent (SGD) principle of noisily approximating a gradient step based on a batch subset of the entire data set. During a learning step, first, a batch $\mathcal{B} = \{(\cdot)_b \mid b \sim \mathcal{U}(\mathbf{0}, \bar{\mathbf{B}})\}$ of $|\mathcal{B}| = B$ tuples are drawn from the memory buffer of size $\bar{B}$. Next, the CrNNs $\hat{Q}$ are updated. A loss is calculated as follows,

$$\mathcal{L}_{\hat{Q}} = \frac{1}{B} \sum_{b \in \mathcal{B}} (\hat{Q}(\mathbf{z}_b, \mathbf{a}_b) - R_b)^2 + \alpha_{\hat{Q}} \|\theta_{\hat{Q}}\|^2, \quad (10)$$

where the first term describes the mean square error in sum rate estimation and the second term is a weight regularization that is discussed later. $\alpha_{\hat{Q}}$ is a scaling term. A SGD-like update is performed to minimize this batch loss. Next, the AcNN is updated. Its loss contains three sum terms:

$$\mathcal{L}_{\mu} = \frac{1}{B} \sum_{b \in \mathcal{B}} -\hat{Q}(\mathbf{z}_b, \mu(\mathbf{z}_b)) \quad (11)$$

$$+ \frac{1}{B} \sum_{b \in \mathcal{B}} \exp(\alpha_e) \log(\pi(\mu(\mathbf{z}_b))) \quad (12)$$

$$+ \alpha_{\mu} \|\theta_{\mu}\|, \quad (13)$$

where α_e, α_{μ} are scaling terms. Recalling that the precoder is sampled from a distribution parameterized by the outputs of the AcNN, $\pi(\cdot)$ is the probability of the sampled precoding, given this distribution. The first term calls the optimization to maximize the estimated sum rate. The second term encourages the optimization to increase the output variance where it does not foresee gains in sum rate from keeping the variance tight. It thereby encourages the AcNN to explore more thoroughly where no good solution has been found yet. The third term is, again, a weight regularization. If multiple CrNN are used, we take the minimum sum rate estimate per tuple for a conservative estimate.

In the following section, we discuss specific implementation choices such as the weight regularization, and then proceed to evaluate the learned precoder and the two benchmarks.

V. EVALUATION

Here, we discuss specific implementation details that we found crucial in learning a precoder. We then present performance comparisons and discuss the relative advantages and disadvantages of the precoders under study. The full code implementation, the trained models, their training configurations and supplemental figures are available online [13]. Table I lists a selection of important system and learning parameters.

A. Implementation Details

As discussed in the previous section, we find rescaling the network inputs to be highly beneficial for fast and stable learning, in accordance with theoretical literature [10]. It could be argued that taking a large number of samples to find the population means and scales before starting to learn is sample

TABLE I
SELECTED PARAMETERS

Noise Power σ_n^2	6e-13 W	Transmit Power P	100 W
Satellite Altitude d_0	600 km	Antenna Nr. N	{10, 16}
User Nr. K	3	Overall Sat. Gain	20 dBi
Wavelength λ	15 cm	Gain per User G_{Usr}	0 dBi
Inter-Ant.-Distance d_n	$3\lambda/2$	SGD Optimizer	Adam
Training batch size B	1024	Init. LR CrNN, AcNN	1e-4, 1e-5
Learning Buffer Size	100 000	Inference / Learning	10 : 1
L2 Scales $\alpha_{\hat{Q}}, \alpha_{\mu}$	0.1	log Entropy Scale α_e	Var.

inefficient and undesirable, however, doing a hypothesis test at a significance level of 5 %, we find the statistics of the population to be approximable within $\pm 10\%$ by taking just 100 samples. For similar reasons, rescaling values not just at the input but also between network layers is desirable and proves beneficial, with intermediate Batch Normalization layers [14] being the most common approach. Our NN implementations correspondingly use four fully connected layers (512 nodes each) stacked alternatingly with Batch Normalization layers.

Another critical change to our prior work is the addition of weight regularization terms in (10), (11). Weight regularization is another standard method, the reasons for its positive influence still being under intense scrutiny [15]. We find weight regularization to be specifically favorable for training in the presence of error, which intuitively can be interpreted as stopping a learning step from committing overmuch to a single batch of channel realizations. In a similar vein, we significantly increase the number of experiences held in the memory buffer compared to our earlier work ($10k \rightarrow 100k$) to provide a more rich set of training experiences to sample from. To compensate, we also adjust the ratio of inference steps per learning step ($1 : 1 \rightarrow 10 : 1$) so that the age of the oldest experience in the buffer stays the same, as per [16].

Overall, we find that the most significant parameter choice is the SGD learning rate, which we keep variable with a Cosine Decay schedule in the area of 1e-4, 1e-5 for CrNN, AcNN respectively. For the detailed configuration we again refer to the repository [13]. We also highlight that, considering practicality, hyper parameter search and training time are limited, thus, our trained models certainly do not present the optimum solution. Training is performed until no significant performance increase is observed, up to around 1e6 to 14e6 simulation steps t depending on the scenario.

B. Evaluation Design

Each simulation will assume a certain mean user distance $\bar{D}_{\text{Usr}}$. For each simulation step t , each user will be assigned a new position uniform randomly within $\pm \bar{D}_{\text{Usr}}/2$ around their mean position, and channel conditions will be updated according to Section II. We investigate the following three scenarios: a) $N = 10$ satellite antennas, mean user distance $\bar{D}_{\text{Usr}} = 100$ km; b) $N = 16$, $\bar{D}_{\text{Usr}} = 100$ km; c) $N = 16$, $\bar{D}_{\text{Usr}} = 10$ km. Before evaluation, we train two learned precoders on each scenario: 1) trained with perfect CSIT, marked with blue square e.g., pSAC1; 2) trained at error

bound $\Delta\varepsilon = 0.05$, marked with green cross, e.g., rSAC1. Evaluations are repeated with 1000 Monte Carlo iterations to account for the stochastic elements of the simulation design, we evaluate the mean performance and its standard deviation.

C. Results

We first explore scenario a), where a mean user distance $\bar{D}_{\text{Usr}} = 100$ km ensures, on average, good spatial partitioning of the users and $N = 10$ satellite antennas will produce wide beams relative to user distances. We train two precoders, pSAC1 with perfect CSIT and rSAC1 with erroneous CSIT, and then compare their mean performance when evaluated on increasingly unreliable CSIT. Fig. 3 presents the mean performance at increasingly large settings of error bound $\Delta\varepsilon$ for the two trained schedulers as well as the MMSE and robust rSLNR precoders from Section III. As expected, all precoders' performances are impacted by increasingly unreliable CSIT. Precoders MMSE, robust rSLNR and pSAC1 achieve comparable performance for perfect CSIT ($\Delta\varepsilon = 0.0$), while rSAC1, having only encountered unreliable CSIT during training, has adopted a strategy that does not scale as well with reliable information. On the other hand, rSAC1 shows the least performance degradation as the CSIT becomes increasingly unreliable. It takes the performance lead at its training point of $\Delta\varepsilon = 0.05$, with the performance gap increasing thereafter. The MMSE precoder, expectedly, shows the worst performance with unreliable CSIT, while the robust rSLNR and pSAC1 precoders are closely matched. This result might, however, be slightly misleading, as robust rSLNR and pSAC1 have adopted significantly different strategies, which we will see in the following.

We repeat this experiment for scenario b), where the increased number of $N = 16$ satellite antennas allows for more narrow beams. This enables better user separation even at close user distances, at the cost of more severe performance drops when a beam is missteered. The results are displayed in Fig. 4 and follow the same trajectory as scenario a), though we see that the narrower beams lead to a more pronounced performance drop as the unreliability $\Delta\varepsilon$ increases. In order to understand how each precoder achieves their robustness, we take a look at their beam patterns for a specific simulation realization. Fig. 5 compares the beam patterns of the rSLNR and our learned precoder pSAC2 with the MMSE precoder. We select this plot as a representative for the precoders' behavior, though we highlight that it depicts just one realization of user positions, error values, large scale fading. Further beam patterns are provided in [13]. Black dots and dotted lines represent the true user positions, whereas the erroneously estimated positions can be discerned by the placement of the MMSE precoder's beams. We observe in the top figure that, compared to the MMSE precoder, the robust rSLNR precoder achieves its robustness by trading beam height in favor of beam width, covering a larger area. On the contrary, the pSAC3 precoder in the bottom figure, not having encountered unreliable information during training, has no incentive to opt for wider beams. Nevertheless, we observe that it achieves robustness by two other factors: 1) power allocation among the

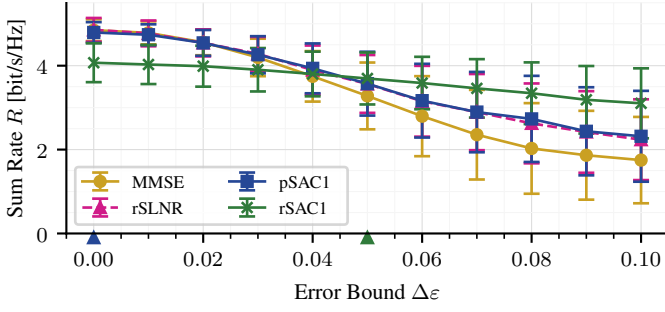


Fig. 3. Scenario a), $N = 10$ satellite antennas, $\bar{D}_{\text{usr}} = 100$ km mean user distance. Testing precoders mean performance with increasing error bounds $\Delta\epsilon$. pSAC1 is trained with perfect CSIT, rSAC1 is trained at $\Delta\epsilon = 0.05$. Markers on the horizontal axis show the error bound that pSAC1 resp. rSAC1 were trained at.

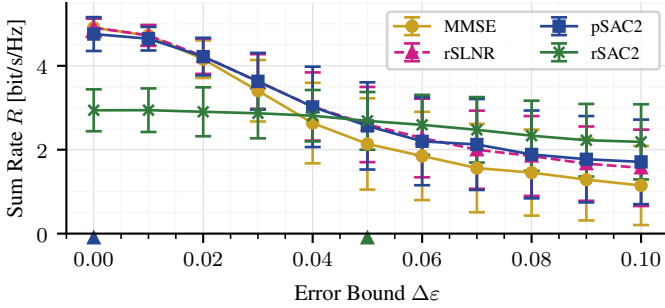


Fig. 4. Scenario b), $N = 16$ satellite antennas, $\bar{D}_{\text{usr}} = 100$ km mean user distance. Testing precoders mean performance with increasing error bounds $\Delta\epsilon$. pSAC2 is trained with perfect CSIT, rSAC2 is trained at $\Delta\epsilon = 0.05$. Markers on the horizontal axis show the error bound that pSAC2 resp. rSAC2 were trained at.

different user's beams; 2) better user tracking. We report that it achieves the second factor by exploiting the power fading information of the CSIT. In the depicted realization, the center user's large scale fading is near one, hence, the power fading of the CSIT closely corresponds to a certain satellite-to-user distance that the learned precoder uses to fine-tune its beam positioning. In Fig. 6, we repeat this comparison for the rSAC2 precoder that was trained for robustness to severe errors. We observe that this scheduler makes use of more irregular beam shapes to cover wide areas.

Finally, in scenario c), we study a case in which the MMSE and rSLNR precoders are not sum rate optimal due to very close average user positioning ($\bar{D}_{\text{usr}} = 10$ km) relative to the beam width. Fig. 7 displays again a performance sweep over increasingly unreliable CSIT for this scenario. We see the learned precoders pSAC3, rSAC3 attain greater sum rate performances, achieved by simply not allocating any power to the center user such that it does not interfere with the other two users. We also see that the robust rSLNR precoder, with its preassumptions violated by the high spatial coupling of user channels, is not able to provide robustness in this scenario even compared to the MMSE precoder. The learned precoders achieve very high degrees of robustness, though we must qualify this result stemming from the erroneous channel realizations being much larger than the variation through user

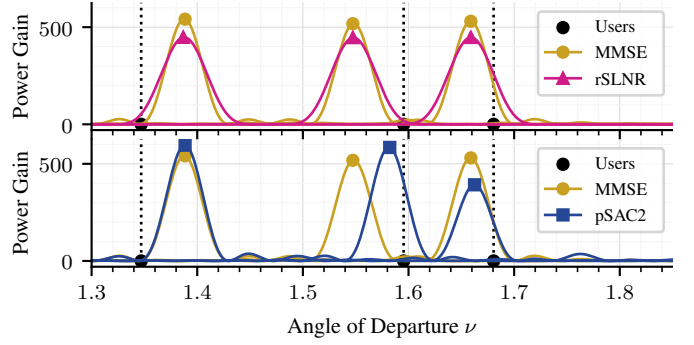


Fig. 5. Beam patterns for one specific simulation realization of scenario b). Curves represent the precoding vectors of the different users at different AoDs ν in linear scale. Sum rates R achieved: MMSE: 0.83 bit/s/Hz, rSLNR: 1.33 bit/s/Hz, pSAC2: 2.15 bit/s/Hz.

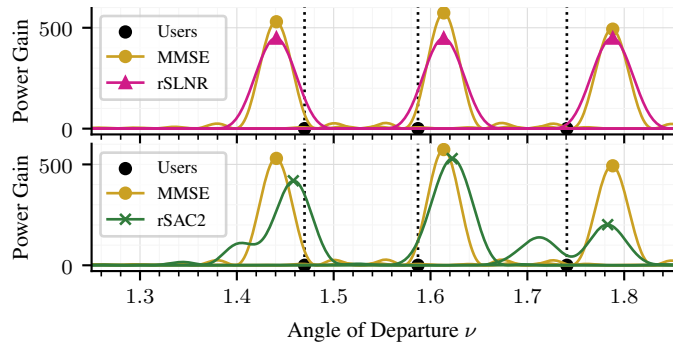


Fig. 6. Beam patterns for one specific simulation realization of scenario b). Curves represent the precoding vectors of the different users at different AoDs ν in linear scale. Sum rates R achieved: MMSE: 0.86 bit/s/Hz, rSLNR: 1.54 bit/s/Hz, rSAC2: 2.07 bit/s/Hz.

positioning in this scenario. The learned precoders exploit this by discarding implausible CSIT, which we do not expect to be possible to the same degree under real circumstances.

In summary, learned precoders offer high flexibility, adjusting to various combinations of user constellations, satellite configurations and error influences, while achieving strong performance. Analytical precoders, while perhaps more predictable, can be complex and costly to solve mathematically and may suffer greatly when the real conditions do not match those that the precoder was modeled on.

VI. CONCLUSIONS

In this paper we studied the influence of imperfect user position knowledge on SDMA precoding in LEO satellite downlink scenarios. Our goal was to design a robust precoding approach that manages inter-user interference for arbitrary error sources and channels. Using data-driven deep RL via the SAC algorithm, we built learned precoders that obtained high performance in terms of both achievable rate and robustness to positioning errors. We qualified these results in comparison to two analytical benchmark precoders, the conventional MMSE precoder as well as a robust precoder that leverages stochastic channel information. Their flexibility at high performance could make learned satellite precoding algorithms an attractive candidate for use in 6G and beyond.

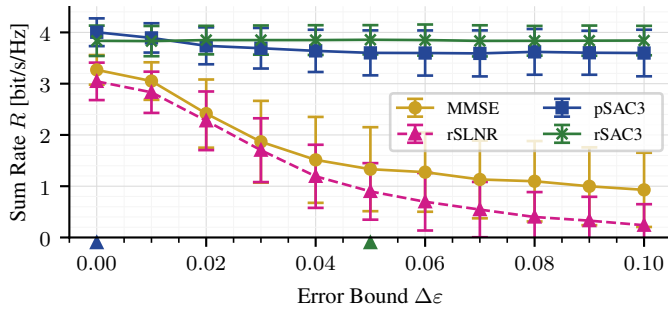


Fig. 7. Scenario c), $N = 16$ satellite antennas, $\bar{D}_{\text{Usr}} = 10$ km mean user distance. Bad conditions for MMSE and rSLNR precoders. Testing precoders mean performance with increasing error bounds $\Delta\epsilon$. pSAC3 is trained with perfect CSIT, rSAC3 is trained at $\Delta\epsilon = 0.05$. Markers on the horizontal axis show the error bound that pSAC3 resp. rSAC3 were trained at.

REFERENCES

- [1] 3GPP TR 38.863, “Technical specification group radio access network; solutions for nr to support non-terrestrial networks (NTN): Non-terrestrial networks (NTN) related RF and co-existence aspects (release 17),” Sep. 2022.
- [2] I. Leyva-Mayorga, B. Soret, M. Röper, D. Wübben, B. Matthiesen, A. Dekorsy, and P. Popovski, “LEO Small-Satellite Constellations for 5G and Beyond-5G Communications,” *IEEE Access*, vol. 8, pp. 184 955–184 964, 2020.
- [3] Z. Qu, G. Zhang, H. Cao, and J. Xie, “LEO Satellite Constellation for Internet of Things,” *IEEE Access*, vol. 5, pp. 18 391–18 401, 2017.
- [4] M. Á. Vázquez, M. B. Shankar, C. I. Kourogiorgas, P.-D. Arapoglou, V. Icolari, S. Chatzinotas, A. D. Panagopoulos, and A. I. Pérez-Neira, “Precoding, Scheduling and Link Adaptation in Mobile Interactive Multibeam Satellite Systems,” *IEEE J. Sel. Areas Commun.*, vol. 36, no. 5, pp. 971–980, 2018.
- [5] Y. Liu, Y. Wang, J. Wang, L. You, W. Wang, and X. Gao, “Robust Downlink Precoding for LEO Satellite Systems With Per-Antenna Power Constraints,” *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10 694–10 711, 2022.
- [6] M. Röper, B. Matthiesen, D. Wübben, P. Popovski, and A. Dekorsy, “Robust Precoding via Characteristic Functions for VSAT to Multi-Satellite Uplink Transmission,” *arXiv:2301.12973*, 2023.
- [7] A. Schröder, M. Röper, D. Wübben, B. Matthiesen, P. Popovski, and A. Dekorsy, “A Comparison between RSMA, SDMA, and OMA in Multibeam LEO Satellite Systems,” in *26th WSA & 13th SCC*, 2023.
- [8] S. Gracla, A. Schröder, M. Röper, C. Bockelmann, D. Wübben, and A. Dekorsy, “Learning Model-Free Robust Precoding for Cooperative Multibeam Satellite Communications,” in *2023 IEEE ICASSP*, 2023.
- [9] H. Dahrouj, R. Alghamdi, H. Alwazani, S. Bahanshal, A. A. Ahmad, A. Faisal, R. Shalabi, R. Alhadrami, A. Subasi, M. T. Al-Nory *et al.*, “An Overview of Machine Learning-Based Techniques for Solving Optimization Problems in Communications and Signal Processing,” *IEEE Access*, vol. 9, 2021.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [11] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, “Soft Actor-Critic Algorithms and Applications,” *arXiv:1812.05905*, Jan. 2019.
- [12] S. Chatzinotas, G. Zheng, and B. Ottersten, “Energy-efficient MMSE beamforming and power allocation in multibeam satellite systems,” in *IEEE ASILOMAR*, 2011.
- [13] A. Schröder and S. Gracla, “Learning Beamforming 2,” https://github.com/Steffengra/2310_beamforming_learner_2, 2023.
- [14] R. Balestriero and R. G. Baraniuk, “Batch Normalization Explained,” Sep. 2022, *arXiv:2209.14778*.
- [15] M. Andriushchenko, F. D’Angelo, A. Varre, and N. Flammarion, “Why Do We Need Weight Decay in Modern Deep Learning?” Oct. 2023, *arXiv:2310.04415*.
- [16] W. Fedus, P. Ramachandran, R. Agarwal, Y. Bengio, H. Larochelle, M. Rowland, and W. Dabney, “Revisiting Fundamentals of Experience Replay,” in *37th PMLR*, 2020.